Московский Физико–Технический Институт

Численные методы
Задание №12

ФИО: Лазарь Владислав Игоревич
Группа: Б01-207а
Почта: lazar.v@phystech.edu

Долгопрудный, 2026 г.

Введение

Целью работы является реализация и оптимизация функции вычисления натурального логарифма одинарной точности (`logf`) для архитектуры `x86_64` с использованием векторных расширений `AVX2` (`AVX512` у меня нет). Основной акцент сделан на увеличении пропускной способности (`throughput`) при сохранении точности вычислений (ошибка не более `3.5 ULP`).

Код с реализацией можно посмотреть в `.../07-MathFunctions/logf_avx2.c`.

Реализация

Для реализации был выбран алгоритм табличной редукции аргумента с последующей аппроксимацией полиномом.

1. Редукция аргумента

Любое нормализованное число x представляется в виде $x = m \cdot 2^n$. Тогда:

$$\ln(x) = n \ln(2) + \ln(m).$$

Для уменьшения диапазона мантиссы $m \in [1, 2)$ используется таблица обратных значений $R_i \approx \frac{1}{m}$. Вычисляется остаток $r = m \cdot R_i - 1$, который близок к нулю. Итоговая формула:

$$\ln(x) \approx n \ln(2) + \mathrm{Poly}(r) - \ln(R_i).$$

Значения $T_i = -\ln(R_i)$ заранее вычисляются и хранятся в таблице.

2. Полиномиальная аппроксимация

Для вычисления логарифма на малом интервале используется полином 3-й степени. Коэффициенты оптимизированы для минимизации максимальной ошибки на отрезке редукции. Расчет производится по схеме Горнера с использованием инструкций FMA.

3. Оптимизации

В ходе работы были применены следующие техники оптимизации:

- **Векторизация.** Использованы регистры `ymm` и инструкции `_mm256_*` для обработки 8 значений `float` за один такт конвейера.
- **Gather Instructions.** Для быстрого доступа к таблицам R_i и T_i по индексам, извлеченным из мантиссы, применена инструкция `_mm256_i32gather_ps`.
- Для компенсации ошибок округления при больших значениях n, константа $\ln(2)$ разбита на две части (`LOG2_HI` и `LOG2_LO`).

Тестирование

Код с тестированием можно посмотреть в `.../08-MathFunctionsTesting/src/test_vector.c`.

Для подтверждения корректности и оценки точности реализованного алгоритма были использованы те же методы тестирования, что и для скалярной версии логарифма — проверка граничных значений и интервальное тестирование.

Оценка точности

Точность вычислений оценивалась в единицах `ULP`. Для этого результат работы векторной функции `logf_avx2` сравнивался с эталонным значением, вычисленным функцией `log()` из библиотеки `math.h` с двойной точностью (`double`).

Расчет ULP производился по формуле из лекции №8:

$$\mathrm{ulp}_{\mathrm{error}} = \frac{|\mathrm{actual} - \mathrm{expected}|}{2^{\exp(A)-23}}.$$

Интервальное тестирование

Был проведен анализ точности на всем диапазоне нормализованных чисел `float`. Весь диапазон был разбит на 253 интервала, соответствующих значениям экспоненты $e \in [-126, 126]$. Внутри каждого интервала $[2^e, 2^{e+1})$ проверялось по 10 000 равномерно распределенных точек.

Результаты тестирования

x	std::log(x)	logf_avx2(x)
5.0000	-0.693147	-0.693147
7.0700	-0.346725	-0.346725
1.4140	0.346423	0.346423
2.0000	0.693147	0.693147
10.0000	2.302585	2.302585
1.0000	0.000000	0.000000
1.1755	-87.336545	-87.336548
3.4028	88.722839	88.722839

Результаты верификации на контрольных точках

По результатам интервального тестирования (допустимый порог — 3.5 `ULP`):

- **Максимальная наблюдаемая ошибка:** 2.11 `ULP`.
- **Количество успешно пройденных интервалов:** 253 из 253.

Сравнение производительности

Код с сравнением производительности можно посмотреть в `.../11-MathFunctionsOptimization_1/`.

Замеры проводились на векторе размером 10^5 элементов. Для векторных реализаций приводится значение `throughput`.

Реализация	**Latency (CPE)**	**Throughput (CPE)**
`std::log`	36.56 ± 0.07	10.29 ± 0.07
`logf_scalar`	119.00 ± 0.31	34.77 ± 0.09
`logf_avx2`	10.80*	3.22 ± 0.07
`SLEEF AVX2`	—	3.93 ± 0.06
`Intel MKL HA`	—	3.66 ± 0.07

** Значение `latency` для `logf_avx2` приведено по результатам микротестирования пачек из 8 элементов.*

Анализ результатов

В ходе тестирования было выявлено, что разработанная реализация `logf_avx2` опережает по пропускной способности как стандартную библиотеку (в 3.2 раза), так и высокооптимизированные пакеты `Intel MKL` и `SLEEF`.

Основным фактором, снижающим производительность `Intel MKL` и `SLEEF` в данном сценарии, является использование динамической линковки (`.so`).

Заключение

В ходе работы реализована векторная версия натурального логарифма, оптимизированная для архитектуры `AVX2`. Достигнуто ускорение по пропускной способности в 2.5 раза относительно `std::log` и в 4 раза относительно скалярной реализации. Точность алгоритма на всем диапазоне нормализованных чисел не превышает 2.11 `ULP`, что полностью соответствует требованиям к математическим функциям одинарной точности.